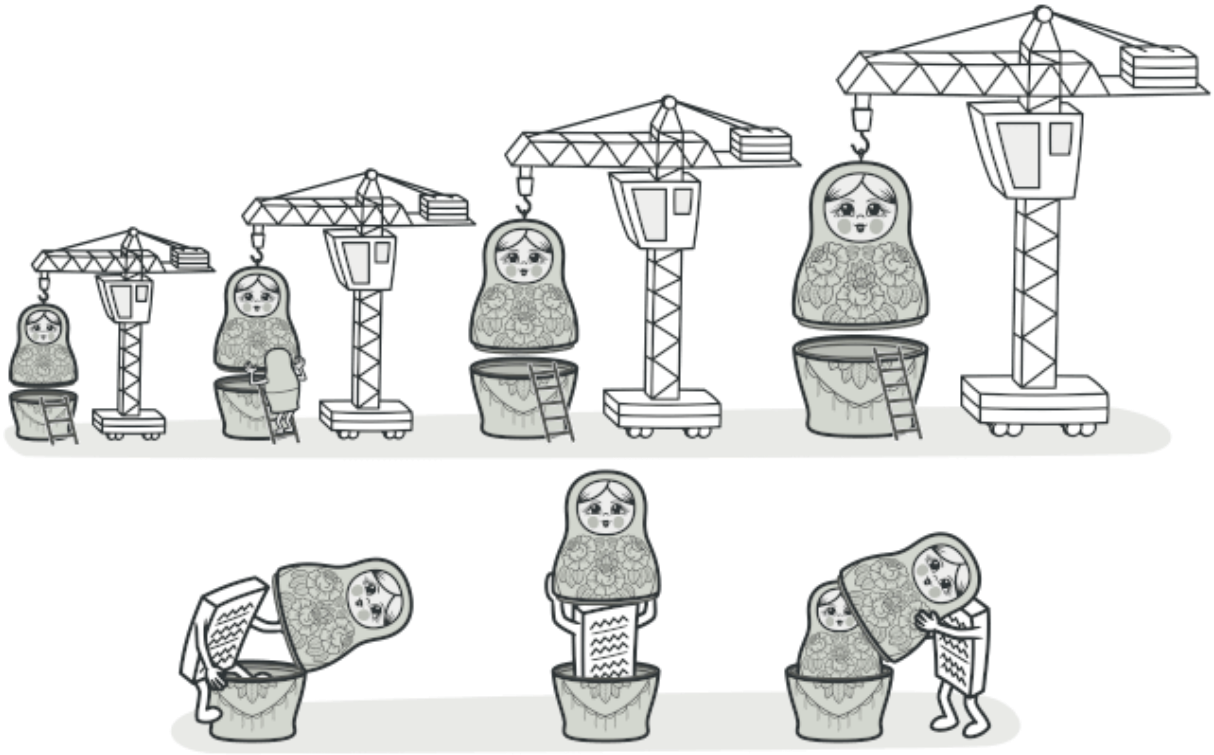
Decorator

Also known as: Wrapper

Absicht

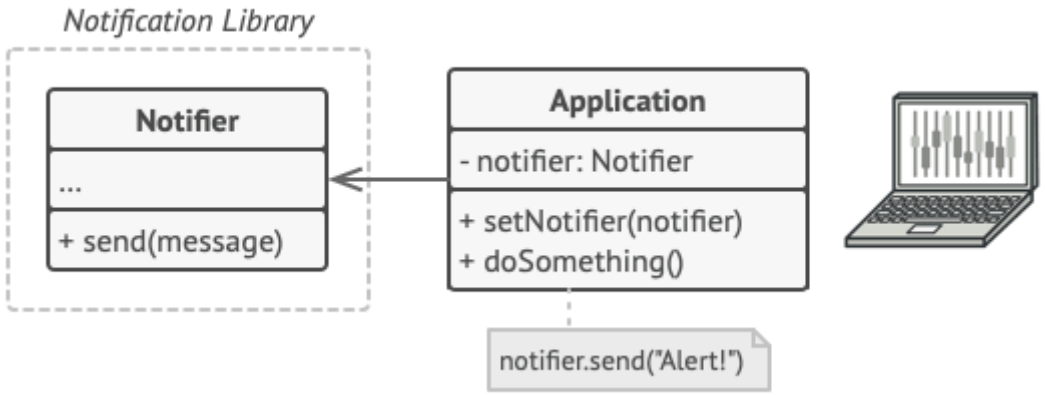
Decorator ist ein strukturelles Entwurfsmuster, mit dem Sie Objekten neue Verhaltensweisen zuordnen können, indem Sie diese Objekte in spezielle Wrapper-Objekte einfügen, die die Verhaltensweisen enthalten.



Problem

Stellen Sie sich vor, Sie arbeiten an einer Benachrichtigungsbibliothek, mit der andere Programme ihre Benutzer über wichtige Ereignisse benachrichtigen können.

Die ursprüngliche Version der Bibliothek basierte auf einer `Notifier`Klasse mit nur wenigen Feldern, einem Konstruktor und einer einzigen `send`Methode. Die Methode konnte ein Nachrichtenargument von einem Client akzeptieren und die Nachricht an eine Liste von E-Mail-Adressen senden, die über ihren Konstruktor an den Notifier weitergeleitet wurden. Eine als Client fungierende Drittanbieter-App sollte das Notifier-Objekt einmalig erstellen und konfigurieren und es dann bei jedem wichtigen Ereignis verwenden.



Ein Programm könnte die Benachrichtigungsklasse verwenden, um Benachrichtigungen über wichtige Ereignisse an einen vordefinierten Satz von E-Mails zu senden.

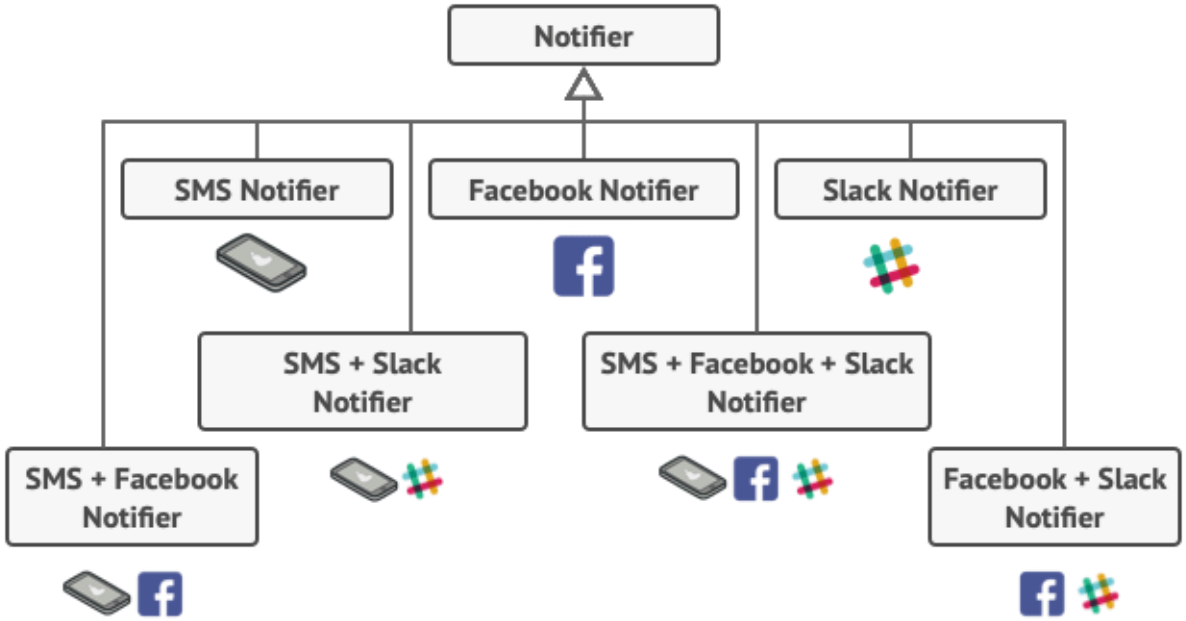
Irgendwann wird einem klar, dass die Bibliotheksnutzer mehr erwarten als nur E-Mail-Benachrichtigungen. Viele möchten bei kritischen Problemen eine SMS erhalten. Andere möchten über Facebook benachrichtigt werden, und die Unternehmensnutzer freuen sich natürlich über Slack-Benachrichtigungen.



Jeder Benachrichtigungstyp wird als Unterklasse eines Benachrichtigers implementiert. Wie schwer kann das sein? Sie haben die `Notifier`Klasse erweitert und die zusätzlichen Benachrichtigungsmethoden in neue Unterklassen gepackt. Nun sollte der Client die gewünschte Benachrichtigungsklasse instanziieren und für alle weiteren Benachrichtigungen verwenden.

Doch dann hat Sie jemand berechtigterweise gefragt: „Warum können Sie nicht mehrere Benachrichtigungsarten gleichzeitig nutzen? Wenn Ihr Haus brennt, möchten Sie doch wahrscheinlich über alle Kanäle informiert werden.“

Sie haben versucht, dieses Problem durch die Erstellung spezieller Unterklassen zu lösen, die mehrere Benachrichtigungsmethoden in einer Klasse kombinierten. Es stellte sich jedoch schnell heraus, dass dieser Ansatz den Code enorm aufblähen würde – nicht nur den Bibliothekscode, sondern auch den Clientcode.



Kombinatorische Explosion von Unterklassen.

Sie müssen eine andere Möglichkeit finden, Benachrichtigungsklassen zu strukturieren, damit ihre Anzahl nicht versehentlich einen Guinness-Rekord bricht.

Lösung

Wenn Sie das Verhalten eines Objekts ändern möchten, denken Sie zunächst an die Erweiterung einer Klasse. Die Vererbung bringt jedoch einige schwerwiegende Nachteile mit sich, die Sie beachten sollten.

- Die Vererbung ist statisch. Sie können das Verhalten eines vorhandenen Objekts zur Laufzeit nicht ändern. Sie können das gesamte Objekt nur durch ein anderes ersetzen, das aus einer anderen Unterklasse erstellt wurde.
- Unterklassen können nur eine übergeordnete Klasse haben. In den meisten Sprachen ist es aufgrund der Vererbung nicht möglich, dass eine Klasse gleichzeitig das Verhalten mehrerer Klassen erbt.

Eine Möglichkeit, diese Einschränkungen zu überwinden, ist die Verwendung von *Aggregation* oder *Komposition* anstelle von *Vererbung* . Beide Alternativen funktionieren fast gleich: Ein Objekt *hat einen* Verweis auf ein anderes und delegiert diesem einen Teil der Arbeit, während bei der Vererbung das Objekt selbst diese Arbeit erledigen kann und das Verhalten von seiner Superklasse erbt .

Mit diesem neuen Ansatz können Sie das verknüpfte „Hilfsobjekt“ einfach durch ein anderes ersetzen und so das Verhalten des Containers zur Laufzeit ändern. Ein Objekt kann das Verhalten verschiedener Klassen nutzen, auf mehrere Objekte verweisen und ihnen alle Arten von Aufgaben delegieren. Aggregation/Komposition ist das Schlüsselprinzip vieler Designmuster, einschließlich Decorator. Kehren wir in diesem Sinne zur Musterdiskussion zurück.

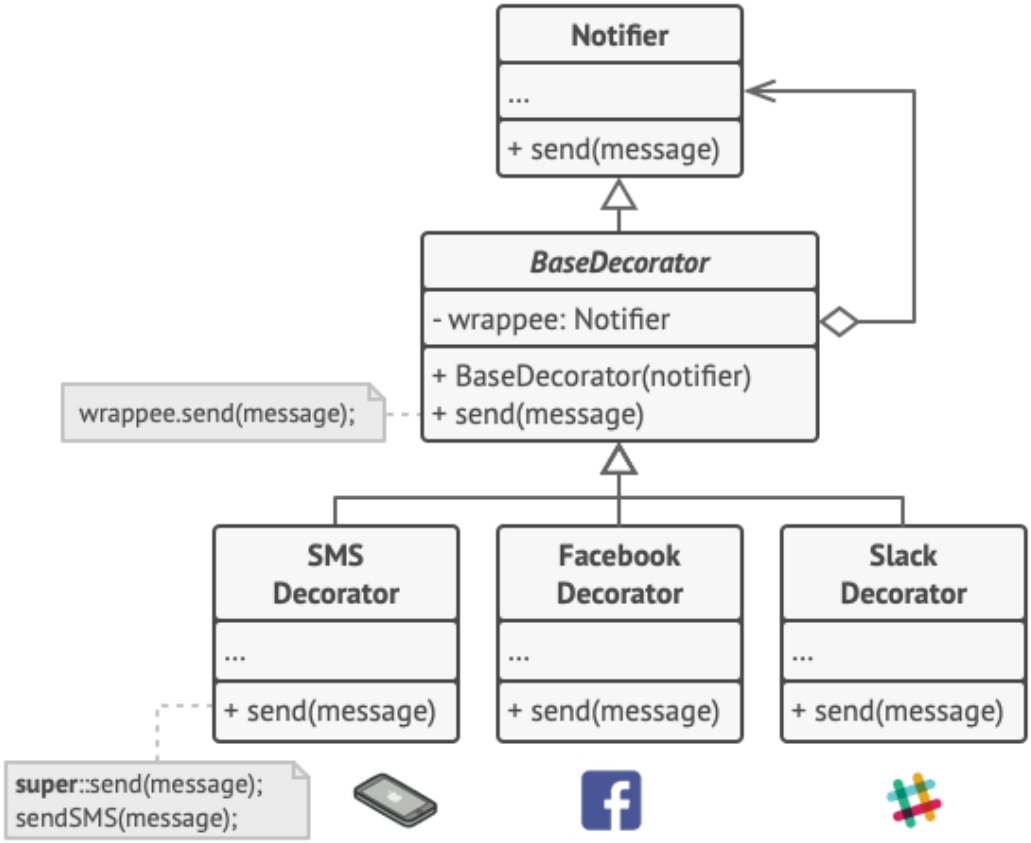


Vererbung vs. Aggregation

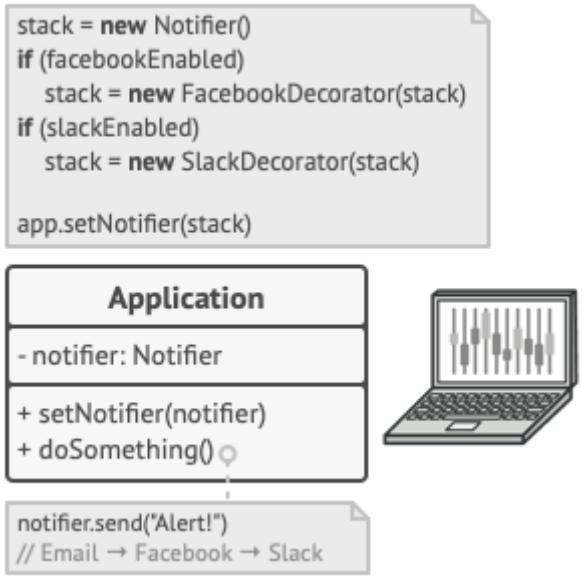
„Wrapper“ ist der alternative Name des Dekorator-Musters, der dessen Grundidee deutlich zum Ausdruck bringt. Ein *Wrapper* ist ein Objekt, das mit einem *Zielobjekt* verknüpft werden kann . Der Wrapper enthält die gleichen Methoden wie das Zielobjekt und delegiert alle eingehenden Anfragen an dieses. Der Wrapper kann das Ergebnis jedoch verändern, indem er vor oder nach der Weiterleitung der Anfrage an das Ziel etwas unternimmt.

Wann wird ein einfacher Wrapper zum eigentlichen Dekorator? Wie bereits erwähnt, implementiert der Wrapper dieselbe Schnittstelle wie das umschlossene Objekt. Aus Clientsicht sind diese Objekte daher identisch. Sorgen Sie dafür, dass das Referenzfeld des Wrappers jedes Objekt akzeptiert, das dieser Schnittstelle folgt. So können Sie ein Objekt in mehreren Wrappern abdecken und ihm das kombinierte Verhalten aller Wrapper hinzufügen.

In unserem Benachrichtigungsbeispiel belassen wir das einfache E-Mail-Benachrichtigungsverhalten in der Basisklasse `Notifier`, wandeln aber alle anderen Benachrichtigungsmethoden in Dekoratoren um.



Verschiedene Benachrichtigungsmethoden werden zu Dekoratoren.
Der Client-Code müsste ein einfaches Benachrichtigungsobjekt in eine Reihe von Dekoratoren einbinden, die den Präferenzen des Clients entsprechen. Die resultierenden Objekte werden als Stapel strukturiert.

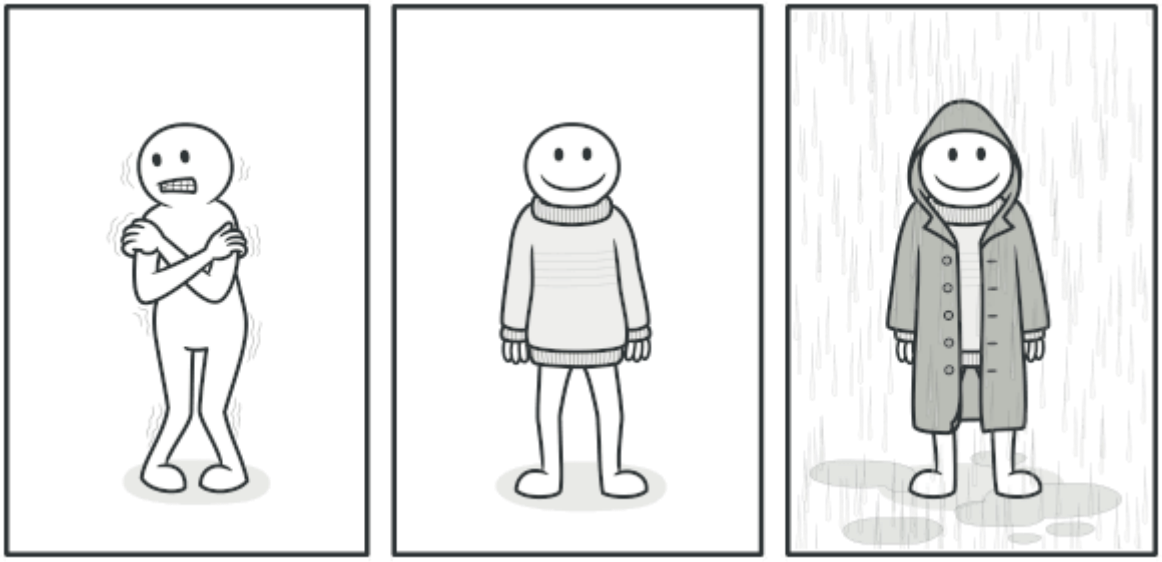


Apps können komplexe Stapel von Benachrichtigungsdekoratoren konfigurieren.
Der letzte Dekorator im Stapel wäre das Objekt, mit dem der Client tatsächlich arbeitet. Da alle Dekoratoren dieselbe Schnittstelle wie der Basis-Notifier implementieren, ist es

für den restlichen Client-Code egal, ob er mit dem „reinen“ oder dem dekorierten Notifier-Objekt arbeitet.

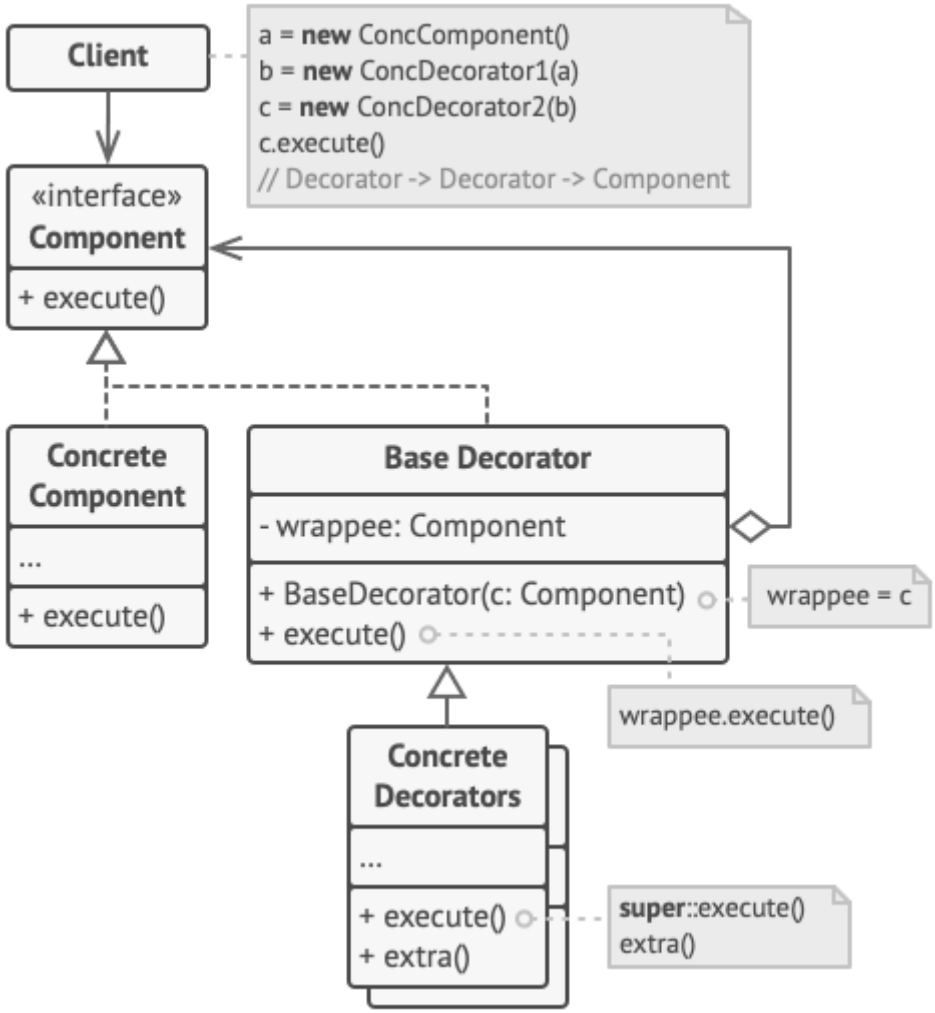
Wir könnten den gleichen Ansatz auch auf andere Verhaltensweisen wie das Formatieren von Nachrichten oder das Erstellen der Empfängerliste anwenden. Der Client kann das Objekt mit beliebigen benutzerdefinierten Dekoratoren dekorieren, solange diese der gleichen Schnittstelle wie die anderen folgen.

Analogie zur realen Welt



Durch das Tragen mehrerer Kleidungsstücke erzielen Sie einen kombinierten Effekt. Das Tragen von Kleidung ist ein Beispiel für den Einsatz von Dekorationen. Wenn dir kalt ist, hülle dich in einen Pullover. Sollte dir auch mit Pullover kalt sein, kannst du eine Jacke darüber tragen. Bei Regen kannst du einen Regenmantel anziehen. All diese Kleidungsstücke „erweitern“ dein Grundverhalten, sind aber kein Teil von dir, und du kannst jedes Kleidungsstück problemlos ausziehen, wenn du es nicht brauchst.

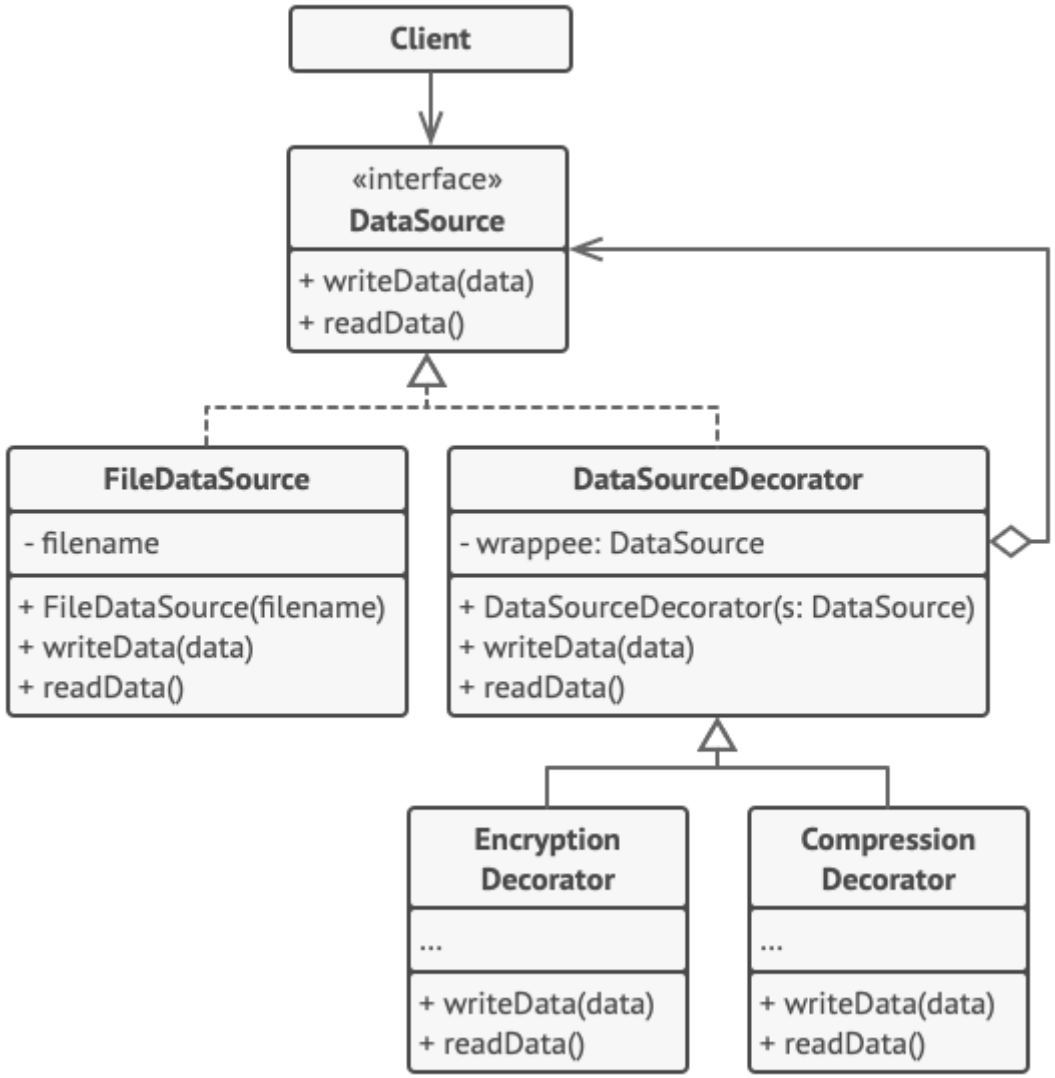
Struktur



1. Die **Komponente** deklariert die gemeinsame Schnittstelle sowohl für Wrapper als auch für gewrappte Objekte.
2. **Konkrete Komponenten** sind eine Klasse von Objekten, die umschlossen werden. Sie definieren das grundlegende Verhalten, das durch Dekoratoren geändert werden kann.
3. Die Klasse **Base Decorator** verfügt über ein Feld zur Referenzierung eines umschlossenen Objekts. Der Typ des Felds sollte als Komponentenschnittstelle deklariert werden, damit es sowohl konkrete Komponenten als auch Dekoratoren enthalten kann. Der Base Decorator delegiert alle Operationen an das umschlossene Objekt.
4. **Konkrete Dekoratoren** definieren zusätzliche Verhaltensweisen, die Komponenten dynamisch hinzugefügt werden können. Konkrete Dekoratoren überschreiben Methoden des Basisdekorators und führen ihr Verhalten entweder vor oder nach dem Aufruf der übergeordneten Methode aus.
5. Der **Client** kann Komponenten in mehrere Dekoratorebenen einschließen, solange er mit allen Objekten über die Komponentenschnittstelle funktioniert.

Pseudocode

In diesem Beispiel können Sie mit dem **Decorator-** Muster vertrauliche Daten unabhängig vom Code komprimieren und verschlüsseln, der diese Daten tatsächlich verwendet.



Das Beispiel für Verschlüsselungs- und Komprimierungsdekoratoren.

Die Anwendung umschließt das Datenquellenobjekt mit einem Dekoratorpaar. Beide Wrapper ändern die Art und Weise, wie die Daten auf die Festplatte geschrieben und von ihr gelesen werden:

- Kurz bevor die Daten **auf die Festplatte geschrieben** werden , verschlüsseln und komprimieren die Dekoratoren sie. Die ursprüngliche Klasse schreibt die verschlüsselten und geschützten Daten in die Datei, ohne von der Änderung zu wissen.
- Direkt nachdem die Daten **von der Festplatte gelesen** wurden , durchlaufen sie dieselben Dekoratoren, die sie dekomprimieren und dekodieren.

Die Dekoratoren und die Datenquellenklasse implementieren dieselbe Schnittstelle, wodurch sie im Clientcode alle austauschbar sind.

```
// The component interface defines operations that can be
// altered by decorators.
interface DataSource is
    method writeData(data)
    method readData():data

// Concrete components provide default implementations for the
// operations. There might be several variations of these
// classes in a program.
class FileDataSource implements DataSource is
    constructor FileDataSource(filename) { ... }

    method writeData(data) is
        // Write data to file.

    method readData():data is
        // Read data from file.

// The base decorator class follows the same interface as the
// other components. The primary purpose of this class is to
// define the wrapping interface for all concrete decorators.
// The default implementation of the wrapping code might include
// a field for storing a wrapped component and the means to
// initialize it.
class DataSourceDecorator implements DataSource is
    protected field wrappee: DataSource

    constructor DataSourceDecorator(source: DataSource) is
        wrappee = source

    // The base decorator simply delegates all work to the
    // wrapped component. Extra behaviors can be added in
    // concrete decorators.
    method writeData(data) is
        wrappee.writeData(data)

    // Concrete decorators may call the parent implementation of
    // the operation instead of calling the wrapped object
    // directly. This approach simplifies extension of decorator
    // classes.
    method readData():data is
        return wrappee.readData()

// Concrete decorators must call methods on the wrapped object,
// but may add something of their own to the result. Decorators
// can execute the added behavior either before or after the
// call to a wrapped object.
class EncryptionDecorator extends DataSourceDecorator is
    method writeData(data) is
        // 1. Encrypt passed data.
        // 2. Pass encrypted data to the wrappee's writeData
        // method.

    method readData():data is
```

```
        // 1. Get data from the wrappee's readData method.
        // 2. Try to decrypt it if it's encrypted.
        // 3. Return the result.

// You can wrap objects in several layers of decorators.
class CompressionDecorator extends DataSourceDecorator is
    method writeData(data) is
        // 1. Compress passed data.
        // 2. Pass compressed data to the wrappee's writeData
        // method.

    method readData():data is
        // 1. Get data from the wrappee's readData method.
        // 2. Try to decompress it if it's compressed.
        // 3. Return the result.


// Option 1. A simple example of a decorator assembly.
class Application is
    method dumbUsageExample() is
        source = new FileDataSource("somefile.dat")
        source.writeData(salaryRecords)
        // The target file has been written with plain data.

        source = new CompressionDecorator(source)
        source.writeData(salaryRecords)
        // The target file has been written with compressed
        // data.

        source = new EncryptionDecorator(source)
        // The source variable now contains this:
        // Encryption > Compression > FileDataSource
        source.writeData(salaryRecords)
        // The file has been written with compressed and
        // encrypted data.


// Option 2. Client code that uses an external data source.
// SalaryManager objects neither know nor care about data
// storage specifics. They work with a pre-configured data
// source received from the app configurator.
class SalaryManager is
    field source: DataSource

    constructor SalaryManager(source: DataSource) { ... }

    method load() is
        return source.readData()

    method save() is
        source.writeData(salaryRecords)
```

```
// ...Other useful methods...


// The app can assemble different stacks of decorators at
// runtime, depending on the configuration or environment.
class ApplicationConfigurator is
    method configurationExample() is
        source = new FileDataSource("salary.dat")
        if (enabledEncryption)
            source = new EncryptionDecorator(source)
        if (enabledCompression)
            source = new CompressionDecorator(source)

        logger = new SalaryManager(source)
        salary = logger.load()
    // ...
```

Anwendbarkeit

Verwenden Sie das Decorator-Muster, wenn Sie Objekten zur Laufzeit zusätzliche Verhaltensweisen zuweisen müssen, ohne den Code zu beschädigen, der diese Objekte verwendet.
Mit dem Dekorator können Sie Ihre Geschäftslogik in Schichten strukturieren, für jede Schicht einen Dekorator erstellen und zur Laufzeit Objekte mit verschiedenen Kombinationen dieser Logik zusammenstellen. Der Clientcode kann alle diese Objekte gleich behandeln, da sie alle einer gemeinsamen Schnittstelle folgen.

Verwenden Sie das Muster, wenn es umständlich oder nicht möglich ist, das Verhalten eines Objekts durch Vererbung zu erweitern.
Viele Programmiersprachen verfügen über das `final`Schlüsselwort, mit dem eine weitere Erweiterung einer Klasse verhindert werden kann. Für eine finale Klasse besteht die einzige Möglichkeit, das vorhandene Verhalten wiederzuverwenden, darin, die Klasse mit einem eigenen Wrapper nach dem Decorator-Muster zu umschließen.

So implementieren Sie

1. Stellen Sie sicher, dass Ihre Geschäftsdomäne als primäre Komponente mit mehreren optionalen Ebenen darüber dargestellt werden kann.
2. Ermitteln Sie, welche Methoden sowohl der primären Komponente als auch den optionalen Ebenen gemeinsam sind. Erstellen Sie eine Komponentenschnittstelle und deklarieren Sie diese Methoden dort.
3. Erstellen Sie eine konkrete Komponentenklasse und definieren Sie darin das Basisverhalten.
4. Erstellen Sie eine Basis-Dekoratorklasse. Diese sollte ein Feld zur Speicherung einer Referenz auf ein gekapseltes Objekt enthalten. Das Feld sollte mit dem Schnittstellentyp der Komponente deklariert werden, um die Verknüpfung mit konkreten Komponenten und Dekoratoren zu ermöglichen. Der Basis-Dekorator muss die gesamte Arbeit an das gekapselte Objekt delegieren.

5. Stellen Sie sicher, dass alle Klassen die Komponentenschnittstelle implementieren.
6. Erstellen Sie konkrete Dekoratoren, indem Sie sie vom Basisdekorator aus erweitern. Ein konkreter Dekorator muss sein Verhalten vor oder nach dem Aufruf der übergeordneten Methode ausführen (die immer an das umschlossene Objekt delegiert).
7. Der Clientcode muss für die Erstellung von Dekoratoren und deren Zusammenstellung gemäß den Anforderungen des Clients verantwortlich sein.

Für und Wider

- Sie können das Verhalten eines Objekts erweitern, ohne eine neue Unterklasse zu erstellen.
- Sie können einem Objekt zur Laufzeit Verantwortlichkeiten hinzufügen oder daraus entfernen.
- Sie können mehrere Verhaltensweisen kombinieren, indem Sie ein Objekt in mehrere Dekoratoren einbinden.
- *Single-Responsibility-Prinzip* . Sie können eine monolithische Klasse, die viele mögliche Verhaltensvarianten implementiert, in mehrere kleinere Klassen aufteilen.

- Es ist schwierig, einen bestimmten Wrapper aus dem Wrapper-Stapel zu entfernen.
- Es ist schwierig, einen Dekorator so zu implementieren, dass sein Verhalten nicht von der Reihenfolge im Dekoratorstapel abhängt.
- Der anfängliche Konfigurationscode der Ebenen sieht möglicherweise ziemlich hässlich aus.

Beziehungen zu anderen Mustern

- **Adapter bietet eine völlig andere Schnittstelle für den Zugriff auf ein vorhandenes Objekt. Beim Decorator** -Muster hingegen bleibt die Schnittstelle entweder gleich oder wird erweitert. Darüber hinaus unterstützt *Decorator rekursive Komposition, die mit Adapter* nicht möglich ist .
- Mit **Adapter** greifen Sie über eine andere Schnittstelle auf ein vorhandenes Objekt zu. Mit **Proxy** bleibt die Schnittstelle gleich. Mit **Decorator** greifen Sie über eine erweiterte Schnittstelle auf das Objekt zu.
- **Chain of Responsibility** und **Decorator** haben sehr ähnliche Klassenstrukturen. Beide Muster basieren auf rekursiver Komposition, um die Ausführung durch eine Reihe von Objekten zu leiten. Es gibt jedoch einige entscheidende Unterschiede.

  Die *CoR*- Handler können beliebige Operationen unabhängig voneinander ausführen. Sie können die Weiterleitung der Anfrage jederzeit abbrechen. Verschiedene *Dekoratoren* können das Verhalten des Objekts erweitern und gleichzeitig die Konsistenz mit der Basisschnittstelle wahren. Darüber hinaus dürfen Dekoratoren den Anfragefluss nicht unterbrechen.

- **Composite** und **Decorator** haben ähnliche Strukturdiagramme, da beide auf rekursiver Komposition basieren, um eine unbegrenzte Anzahl von Objekten zu organisieren.

  Ein *Decorator* ähnelt einem *Composite* , hat aber nur eine untergeordnete Komponente. Es gibt einen weiteren wesentlichen Unterschied: *Der Decorator* fügt dem umschlossenen Objekt zusätzliche Verantwortlichkeiten hinzu, während *das Composite* lediglich die Ergebnisse seiner untergeordneten Komponenten zusammenfasst.

  Die Muster können jedoch auch zusammenarbeiten: Sie können *Decorator* verwenden , um das Verhalten eines bestimmten Objekts im *Composite-* Baum zu erweitern.

- **Designs, die häufig Composite** und **Decorator** verwenden , profitieren oft von der Verwendung von **Prototype** . Durch die Anwendung des Musters können Sie komplexe Strukturen klonen, anstatt sie von Grund auf neu zu erstellen.
- **Mit Decorator** können Sie die Oberfläche eines Objekts ändern, während Sie **mit Strategy** die Innenseiten ändern können.
- **Dekorator** und **Proxy** haben ähnliche Strukturen, aber sehr unterschiedliche Absichten. Beide Muster basieren auf dem Kompositionsprinzip, bei dem ein Objekt einen Teil der Arbeit an ein anderes delegieren soll. Der Unterschied besteht darin, dass ein *Proxy* den Lebenszyklus seines Serviceobjekts in der Regel selbst verwaltet, während die Komposition von *Dekoratoren* stets vom Client gesteuert wird.